

Technical Project Documentation

Advanced RAG Implementation with LangGraph Orchestration

1. The RAG Chain Logic

The "Chain" refers to the sequential flow of data from ingestion to retrieval. The project implements a robust semantic search chain using HuggingFace and ChromaDB.

The Processing Chain:

PDF Content → Recursive Splitting → Vector Embedding → ChromaDB Storage → Similarity Retrieval

Key Implementation Details:

- **Model:** llama3-8b-8192 via Groq for high-speed inference.
- **Embeddings:** all-MiniLM-L6-v2 for efficient semantic representation.
- **Vector Store:** ChromaDB for persistent, local storage.

2. LangGraph Workflow Visualization

The "Graph" provides a non-linear control structure, allowing for conditional routing and human intervention.

Node Definitions:

- **Retrieve:** Queries the vector database for document context.
- **Generate:** Analyzes context and formulates a customer-friendly response.
- **Escalate:** Triggers when information is missing, routing to a human agent.

3. Complete Source Code Implementation

```
import os
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from groq import Groq

# State Definition
class AgentState(TypedDict):
    query: str
    context: str
    answer: str
    needs_escalation: bool

# Logic Flow
def retrieve_node(state):
    vs = Chroma(persist_directory="./chroma_db",
embedding_function=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2"))
    docs = vs.similarity_search(state['query'], k=3)
    return {"context": "\n".join([d.page_content for d in docs])}

def generate_node(state):
    if not state['context'] or len(state['context']) < 50:
        return {"needs_escalation": True}
    client = Groq(api_key=os.getenv("GROQ_API_KEY"))
    res = client.chat.completions.create(
        messages=[{"role": "user", "content": f"Context: {state['context']}\n\nQ: {state['query']}"}],
        model="llama3-8b-8192"
    )
    return {"answer": res.choices[0].message.content, "needs_escalation": False}

def escalate_node(state):
    print("\n[!] HUMAN INTERVENTION REQUIRED")
    ans = input("Provide manual response: ")
    return {"answer": ans, "needs_escalation": False}

# Build Graph
workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("generate", generate_node)
workflow.add_node("escalate", escalate_node)
```

```
workflow.set_entry_point("retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_conditional_edges("generate", lambda x: "escalate" if
x["needs_escalation"] else END)
workflow.add_edge("escalate", END)

app = workflow.compile()
```

4. Technical Specifications

Component	Technology	Detail
Orchestration	LangGraph	State-based routing
Vector Storage	ChromaDB	Persistence at ./chroma_db
PDF Loading	PyPDF	Text extraction & splitting
Inference	Groq API	Llama 3 (8B) model